DOSSIER DE CONCEPTION

Application de gestion de bibliothèque

Conception fonctionnelle, technique et graphique



Projet : Bibliothèque — Médiathèque en ligne

Document : Dossier de conception, version 1.0

Date : octobre 2026

Technologies : Java 21 · Spring Boot 3.5 · Angular 22 · MariaDB 11.4 · Docker

Sommaire

1 Introduction et démarche

2 Cas d'utilisation

3 Architecture générale

4 Architecture logicielle du backend

5 Modèle de données

6 Dynamique : états et séquences

7 Contrat d'API REST

8 Sécurité

9 Traitements planifiés

10 Conception de l'interface (UX/UI)

11 Déploiement

12 Stratégie de tests et résultats

13 Décisions d'architecture (ADR)

14 Traçabilité et limites

75 opérations d'API	**18** tables (98 contraintes nommées)	**40** écrans conçus
193 classes Java (+ 46 de test)	**129** fichiers Angular (+ 22 de test)	**6** décisions d'architecture

1 Introduction et démarche

1.1 Objet

Ce dossier décrit comment l'application répond au cahier des charges : architecture, modèle de données, contrat d'API, sécurité, interface et déploiement. Il s'appuie sur les livrables du projet (spécifications, maquettes UX, documents d'architecture, ADR, contrat OpenAPI, rapports de développement) et sur le code réellement livré.

1.2 Démarche en six rôles

Rôle	Livrables produits	Emplacement
Analyste métier	7 spécifications : 44 user stories, 41 règles de gestion, 12 exigences non fonctionnelles	`docs/specs/`
Designer UX/UI	40 écrans, parcours, composants, jetons de design, accessibilité	`docs/design/`
Architecte	Modèle de données, migrations Flyway, contrat OpenAPI, 6 ADR	`docs/architecture/`
Développeur fullstack	Code backend et frontend, tests, rapports de développement	`backend/`, `frontend/`, `docs/dev/`
Relecteur de code	Revue de conformité et de sécurité	à venir
Testeur QA	Tests d'intégration et de bout en bout	à venir

1.3 Principes directeurs

- **Le contrat d'abord** : pour chaque domaine, le modèle de données et l'API sont définis avant le code.
- **Les règles en base quand c'est possible** : contraintes `CHECK`, `UNIQUE` et clés étrangères en plus de la validation applicative.
- **Sécurité par construction** : l'adhérent est déduit de la session, jamais d'un paramètre ; tout est refusé par défaut.
- **Traçabilité** : chaque écran, endpoint et test se rattache à une user story (US) et à une règle (RG).

2 Cas d’utilisation

Quatre acteurs utilisent le système. L’administrateur dispose de tous les droits du bibliothécaire, plus l’administration.

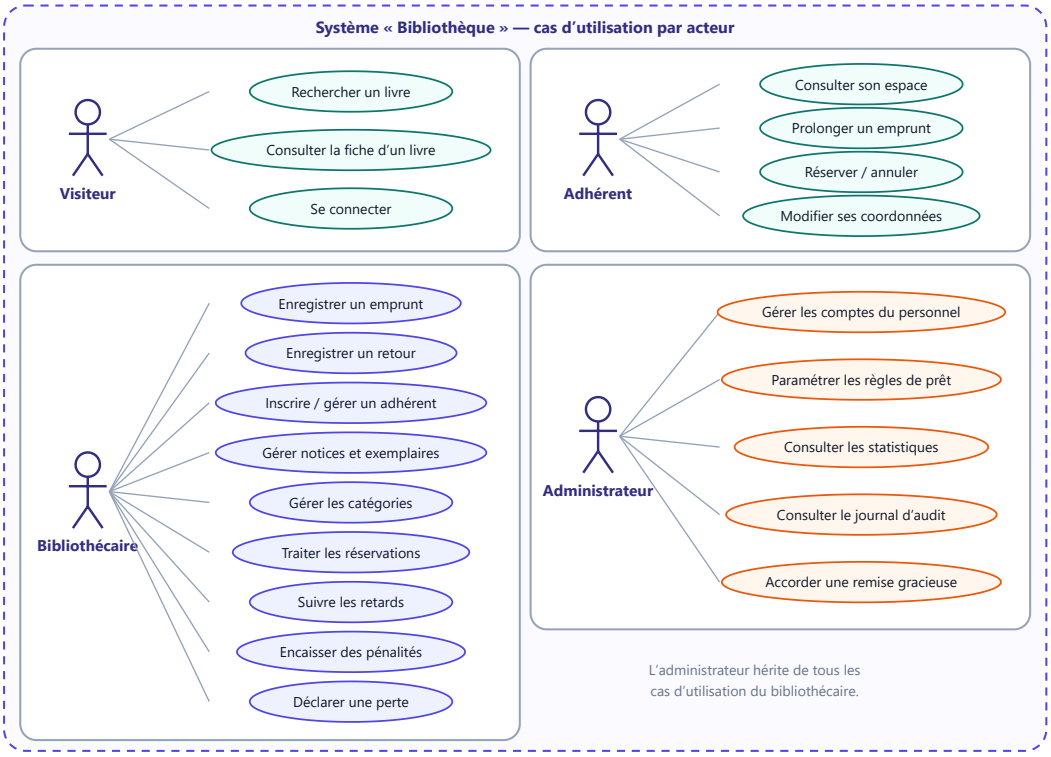


Figure 1 — Diagramme des cas d’utilisation

3 Architecture générale

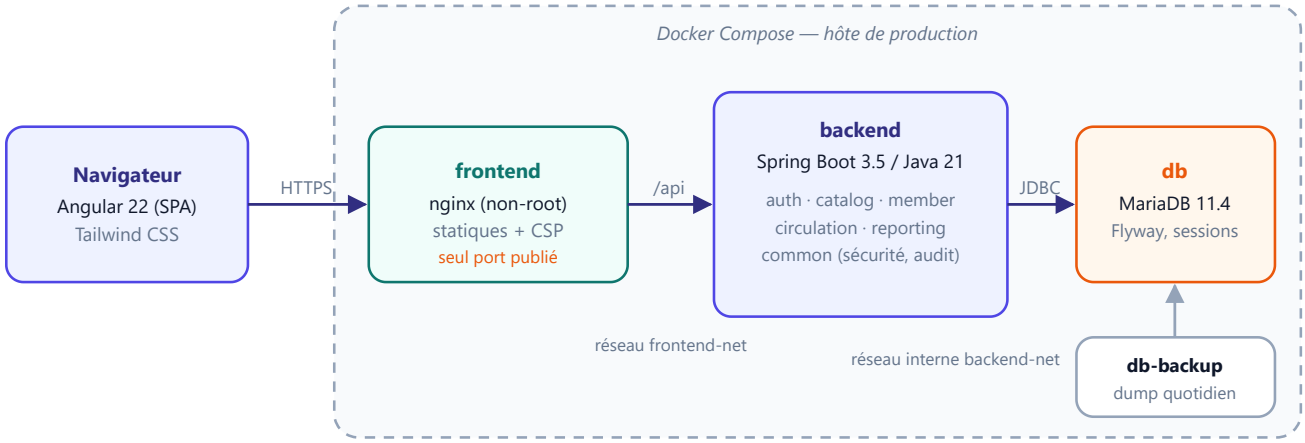


Figure 2 — Architecture de déploiement (Docker Compose)

L'application est une **application web monopage** (Angular) servie par **nginx**, qui relaie les appels `/api` vers un **backend Spring Boot** sur la même origine : aucune configuration CORS n'est nécessaire et le cookie de session reste `SameSite=Strict`. Seul le conteneur frontend publie un port ; la base et le backend restent sur des réseaux internes.

3.1 Pile technique

Couche	Technologie	Rôle
Interface	Angular 22 (standalone, signals, OnPush), TypeScript strict, Tailwind CSS 3.4, police Inter	Écrans public, adhérent et personnel
Serveur web	nginx (image non-root)	Fichiers statiques, relais /api, en-têtes de sécurité (CSP)
Backend	Java 21, Spring Boot 3.5 (Web, Data JPA, Security, Validation), Spring Session JDBC	Logique métier, API REST, sécurité
Données	MariaDB 11.4, Flyway, Hibernate	Persistance, intégrité, sessions
Build	Gradle (Groovy DSL) + wrapper, npm / Angular CLI	Compilation, tests, images
Qualité	JUnit 5, Mockito, AssertJ, ArchUnit, JaCoCo, Testcontainers, Vitest, Cypress	Tests à tous les niveaux
Exploitation	Docker, Docker Compose, sauvegarde mariadb-dump	Déploiement et sauvegarde

4 Architecture logicielle du backend

4.1 Découpage en modules métier

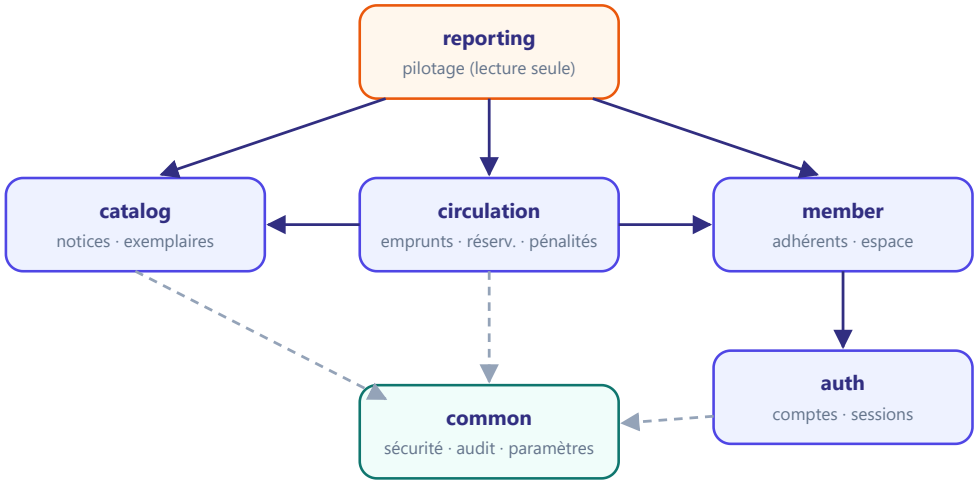


Figure 3 — Modules du backend : flèche pleine = dépendance autorisée (vérifiée par ArchUnit), pointillés = utilisation de common

Module	Domaines couverts	Peut dépendre de
common	Configuration, sécurité, erreurs, pagination, audit, paramètres	—
auth	Comptes, connexion, mots de passe	common
catalog	Notices, auteurs, catégories, exemplaires, recherche	common
member	Adhérents, adhésion, espace adhérent	common, auth
circulation	Emprunts, réservations, pénalités	common, catalog, member
reporting	Tableau de bord, statistiques, journal	tous (lecture seule)

Les échanges entre modules qui iraient « à contre-sens » passent par des **ports** (interfaces) implémentés par le module qui détient la logique : par exemple `MemberCirculation` (déclaré par `member`, implémenté par `EligibilityService` de `circulation`) ou `CopyAllocation` (déclaré par `catalog`, implémenté par `ReservationAllocator`). Ces règles sont vérifiées automatiquement par des tests **ArchUnit**.

4.2 Couches internes

Couche	Contenu	Règle
web	Contrôleurs REST, requêtes validées (records)	N'accède jamais directement aux repositories
service	Logique métier, transactions, vues (records)	Ne renvoie jamais d'entité JPA
repository / domain	Entités JPA, énumérations, requêtes	Ne dépend pas des couches supérieures
job	Traitements planifiés	N'appelle que la couche service

4.3 Gestion des erreurs

Toutes les erreurs suivent le format **ProblemDetail** (RFC 9457) avec un `code` métier stable (ex. `MEMBER_NOT_ELIGIBLE`, `COPY_NOT_AVAILABLE`, `CONCURRENT_MODIFICATION`), la liste des `reasons` en cas de refus multiples et les `fieldErrors` de validation. Le frontend affiche un message français propre à chaque code.

5 Modèle de données

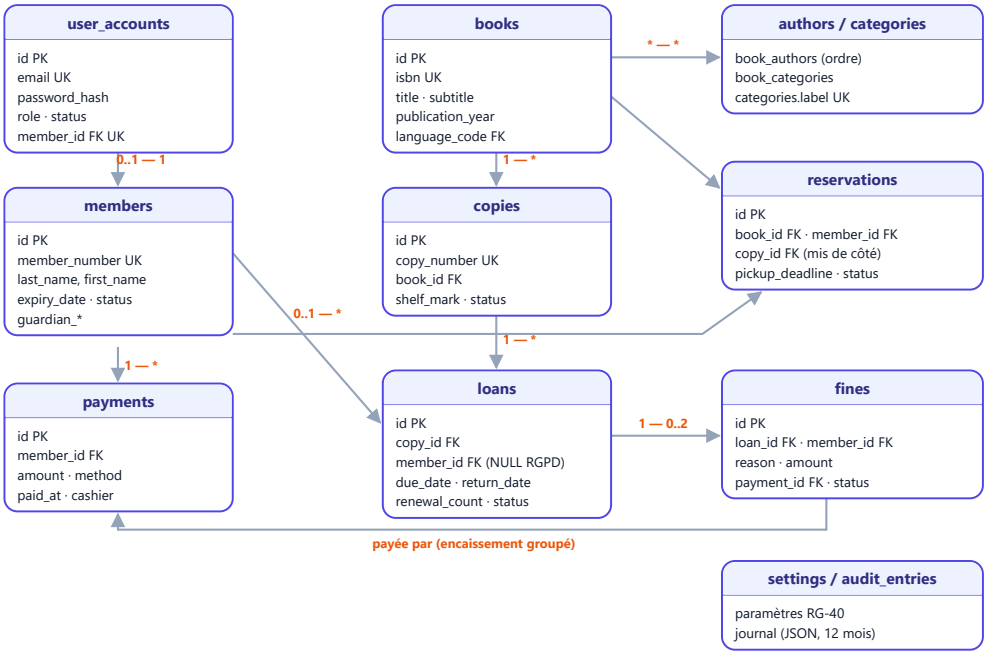


Figure 4 — Modèle de données simplifié (tables principales)

Table	Rôle	Intégrité garantie en base
members	Adhérents	Numéro A + 6 chiffres ; identité obligatoire sauf archivé ; représentant légal complet ou absent
user_accounts	Comptes (tous rôles)	E-mail unique sans tenir compte de la casse ; MEMBER ⇔ adhérent rattaché
books, book_authors, book_categories	Notices	ISBN-13 unique ; année ≥ 1450 ; ordre des auteurs unique
copies	Exemplaires	Numéro à 8 chiffres ; retrait ⇔ motif + date ; jamais supprimés
loans	Emprunts	Un seul emprunt en cours par exemplaire (colonne générée + index unique)
reservations	Réservations	Une seule active par adhérent et notice ; un exemplaire pour une seule mise de côté
fines, payments	Pénalités et encaissements	Une pénalité par emprunt et par motif ; montant > 0 ; payée ⇔ encaissement
settings	Paramètres des règles (RG-40)	Valeur dans les bornes
audit_entries	Journal d'audit	Détail JSON valide ; immuable
SPRING_SESSION*	Sessions HTTP	Schéma Spring Session

Choix notables. Collation `utf8mb4_unicode_ci` (recherche et unicité insensibles à la casse et aux accents) ; numéros fonctionnels par séquences jamais réattribuées ; verrou optimiste (`version`) sur toute table modifiable ; RGPD par effacement des colonnes d'identité à l'archivage ; schéma géré uniquement par Flyway (migrations immuables).

6 Dynamique : états et séquences

6.1 Cycle de vie d'un exemplaire

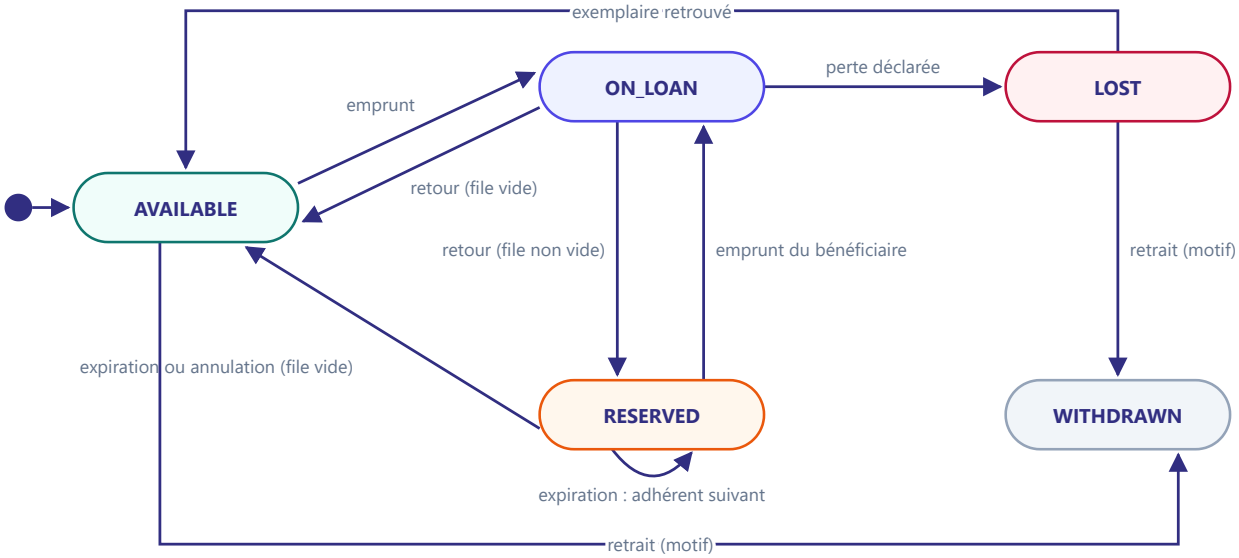


Figure 5 — Diagramme d'états de l'exemplaire

6.2 Enregistrement d'un emprunt au guichet

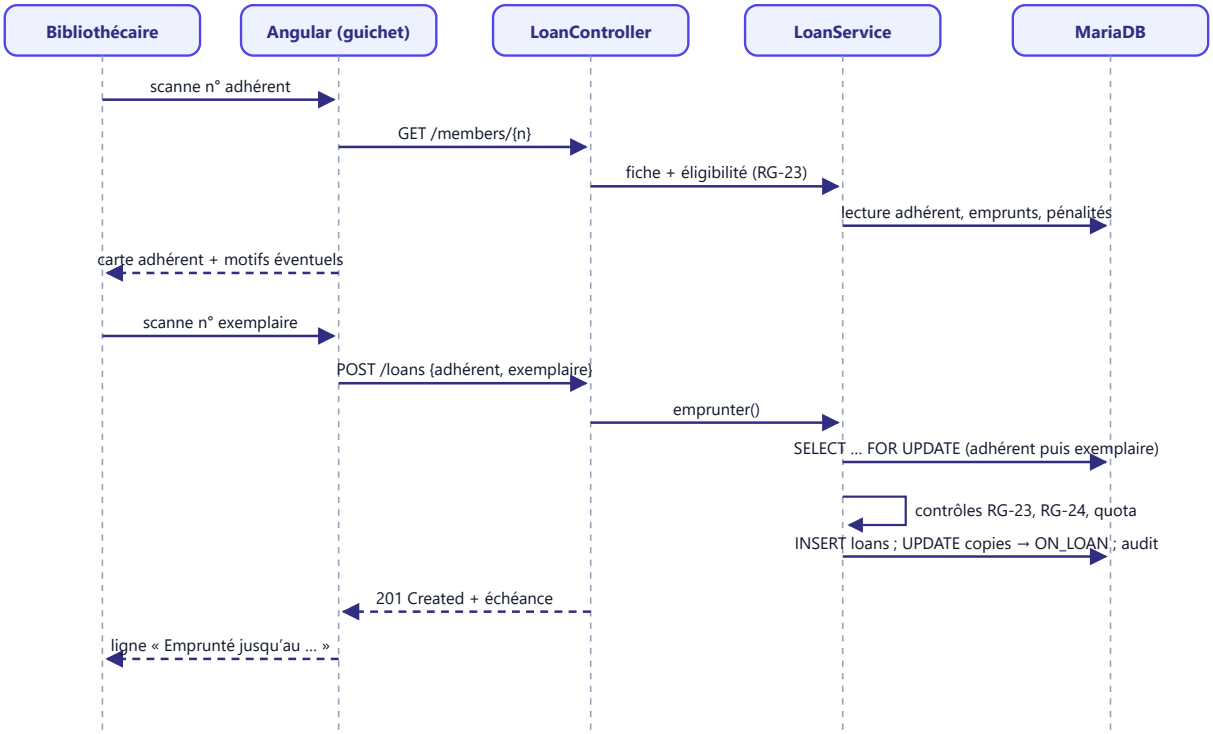


Figure 6 — Diagramme de séquence : emprunt (US-025)

Les verrous de ligne sont toujours pris dans le même ordre (adhérent puis exemplaire) pour éviter les interblocages ; une contrainte d'unicité en base garantit qu'un exemplaire ne peut jamais avoir deux emprunts actifs (NFR-10), même en cas d'opérations simultanées.

7 Contrat d'API REST

Le contrat `docs/architecture/openapi.yaml` (OpenAPI 3.1) est la source de vérité : 75 opérations et 123 schémas. Conventions : préfixe `/api/v1`, ressources au pluriel, identifiants fonctionnels dans les chemins, pagination `?page&size&sort` avec liste blanche des tris, version renvoyée pour le contrôle de concurrence (409 si périmée), espace adhérent sous `/me/**`.

Authentification

Méthode	Chemin	Description
POST	`/auth/login`	Ouvrir une session (US-001)
POST	`/auth/logout`	Fermer la session (US-002)
GET	`/auth/me`	Lire la session courante
POST	`/auth/password`	Changer son mot de passe (US-003, RG-01, RG-05)
GET	`/staff-accounts`	Lister les comptes du personnel (US-004, US-005) — ADMIN
POST	`/staff-accounts`	Créer un compte du personnel (US-004) — ADMIN
GET	`/staff-accounts/{id}`	Lire un compte du personnel (cible de Location) — ADMIN
POST	`/staff-accounts/{id}/deactivation`	Désactiver un compte du personnel (US-005, RG-06) — ADMIN
POST	`/staff-accounts/{id}/reactivation`	Réactiver un compte du personnel (US-005) — ADMIN
POST	`/accounts/{id}/password-reset`	Attribuer un mot de passe temporaire (US-006) — LIBRARIAN (comptes MEMBER), ADMIN

Catalogue

Méthode	Chemin	Description
GET	`/catalog/books`	Rechercher dans le catalogue public (US-007, RG-11)
GET	`/catalog/books/{id}`	Fiche publique d'une notice (US-008)
GET	`/catalog/categories`	Liste des catégories (filtres et saisie)
GET	`/catalog/languages`	Liste des langues
GET	`/books`	Rechercher les notices (gestion, US-007 AC-007-7) — LIBRARIAN, ADMIN
POST	`/books`	Créer une notice (US-009) — LIBRARIAN, ADMIN
GET	`/books/{id}`	Fiche de gestion d'une notice (US-008 AC-008-2) — LIBRARIAN, ADMIN
PUT	`/books/{id}`	Modifier une notice (US-010) — LIBRARIAN, ADMIN
DELETE	`/books/{id}`	Supprimer une notice sans exemplaire (US-011) — LIBRARIAN, ADMIN
POST	`/books/{id}/copies`	Ajouter des exemplaires (US-012) — LIBRARIAN, ADMIN
GET	`/authors`	Suggérer des auteurs (US-009) — LIBRARIAN, ADMIN
POST	`/authors`	Créer un auteur (AC-009-2) — LIBRARIAN, ADMIN
PUT	`/copies/{copyNumber}`	Modifier la cote et l'état d'un exemplaire (US-013) — LIBRARIAN, ADMIN
POST	`/copies/{copyNumber}/withdrawal`	Retirer un exemplaire du fonds (US-013, RG-10) — LIBRARIAN, ADMIN

Méthode	Chemin	Description
GET	`/categories`	Lister les catégories avec leur nombre de notices (US-014) — LIBRARIAN, ADMIN
POST	`/categories`	Créer une catégorie (US-014) — LIBRARIAN, ADMIN
PUT	`/categories/{id}`	Renommer une catégorie (US-014) — LIBRARIAN, ADMIN
DELETE	`/categories/{id}`	Supprimer une catégorie inutilisée (US-014, RG-12) — LIBRARIAN, ADMIN

Adherents

Méthode	Chemin	Description
GET	`/members`	Rechercher des adhérents (US-017) — LIBRARIAN, ADMIN
POST	`/members`	Inscrire un adhérent et créer son compte (US-016) — LIBRARIAN, ADMIN
GET	`/members/to-archive`	Adhérents expirés depuis plus de 3 ans (US-022, RG-19) — LIBRARIAN, ADMIN
GET	`/members/{memberNumber}`	Fiche d'un adhérent avec éligibilité (US-018) — LIBRARIAN, ADMIN
PUT	`/members/{memberNumber}`	Modifier un adhérent (US-019) — LIBRARIAN, ADMIN
POST	`/members/{memberNumber}/renewal`	Renouveler l'adhésion (US-020, RG-15) — LIBRARIAN, ADMIN
POST	`/members/{memberNumber}/suspension`	Suspendre un adhérent (US-021, RG-17) — LIBRARIAN, ADMIN
POST	`/members/{memberNumber}/reactivation`	Réactiver un adhérent suspendu (US-021) — LIBRARIAN, ADMIN
GET	`/members/{memberNumber}/archive-check`	Vérifier l'archivabilité (US-022, RG-18) — LIBRARIAN, ADMIN
POST	`/members/{memberNumber}/archive`	Archiver et anonymiser un adhérent (US-022, RG-18) — LIBRARIAN, ADMIN
GET	`/me/member`	Identité de l'adhérent connecté (US-023) — MEMBER
PUT	`/me/contact`	Modifier mes coordonnées (US-024) — MEMBER

Circulation

Méthode	Chemin	Description
GET	`/members/{memberNumber}/loans`	Emprunts en cours d'un adhérent (ECR-18) — LIBRARIAN, ADMIN
GET	`/members/{memberNumber}/reservations`	Réservations actives d'un adhérent (ECR-18) — LIBRARIAN, ADMIN
GET	`/members/{memberNumber}/fines`	Pénalités impayées d'un adhérent (ECR-18, ECR-34) — LIBRARIAN, ADMIN
GET	`/members/{memberNumber}/history`	Emprunts rendus depuis 4 mois (RG-19) — LIBRARIAN, ADMIN
GET	`/me/loans`	Mes emprunts en cours (US-023) — MEMBER
POST	`/me/loans/{id}/renewal`	Prolonger mon emprunt (US-027) — MEMBER
GET	`/me/reservations`	Mes réservations actives (US-023) — MEMBER
POST	`/me/reservations`	Réserver un livre (US-032) — MEMBER
POST	`/me/reservations/{id}/cancellation`	Annuler ma réservation (US-033) — MEMBER

Méthode	Chemin	Description
GET	`/me/fines`	Mes pénalités impayées (US-023) — MEMBER
GET	`/me/history`	Mon historique sur 4 mois (AC-023-2) — MEMBER
GET	`/me/books/{bookId}/reservation`	Ma situation de réservation sur une notice (ECR-09, AC-034-3) — MEMBER
POST	`/loans`	Enregistrer un emprunt (US-025) — LIBRARIAN, ADMIN
POST	`/returns`	Enregistrer un retour (US-026, US-037) — LIBRARIAN, ADMIN
POST	`/loans/{id}/renewal`	Prolonger un emprunt au guichet (AC-027-7) — LIBRARIAN, ADMIN
GET	`/loans/overdue`	Emprunts en retard (US-028) — LIBRARIAN, ADMIN
GET	`/loans/{id}/loss-preview`	Prévisualiser les pénalités de perte (ECR-27) — LIBRARIAN, ADMIN
POST	`/loans/{id}/loss`	Déclarer un exemplaire perdu (US-029, RG-28) — LIBRARIAN, ADMIN
POST	`/copies/{copyNumber}/found`	Remettre en circulation un exemplaire retrouvé (US-030) — LIBRARIAN, ADMIN
GET	`/copies/{copyNumber}/loans`	Historique de circulation d'un exemplaire (US-031) — LIBRARIAN, ADMIN
GET	`/reservations/preview`	Prévisualiser une réservation pour un adhérent (ECR-31) — LIBRARIAN, ADMIN
POST	`/reservations`	Réserver pour un adhérent (AC-032-8) — LIBRARIAN, ADMIN
POST	`/reservations/{id}/cancellation`	Annuler une réservation (US-033, RG-34) — LIBRARIAN, ADMIN
GET	`/books/{id}/reservations`	File d'attente d'une notice (US-034) — LIBRARIAN, ADMIN
GET	`/reservations/ready`	Réservations prêtes (US-035) — LIBRARIAN, ADMIN
GET	`/reservations/released`	Exemplaires libérés à ranger (US-035, Q-26) — LIBRARIAN, ADMIN
POST	`/payments`	Encaisser des pénalités (US-038, RG-38) — LIBRARIAN, ADMIN
POST	`/fines/{id}/waiver`	Accorder une remise gracieuse (US-039, RG-39) — ADMIN
GET	`/fines/debtors`	Adhérents ayant des pénalités impayées (US-040) — LIBRARIAN, ADMIN

Pilotage

Méthode	Chemin	Description
GET	`/dashboard`	Indicateurs du jour (US-041) — LIBRARIAN, ADMIN
GET	`/statistics`	Statistiques d'un mois (US-042, RG-41) — ADMIN
GET	`/statistics/export`	Exporter les statistiques en tableur (AC-042-3) — ADMIN
GET	`/settings`	Paramètres des règles de prêt (US-043) — ADMIN
PUT	`/settings/{code}`	Modifier un paramètre (US-043, RG-40) — ADMIN
GET	`/audit-entries`	Journal d'audit, lecture seule (US-044) — ADMIN

8 Sécurité

Mesure	Mise en œuvre
Authentification	Session serveur (Spring Session JDBC), cookie `HttpOnly`, `Secure`, `SameSite=Strict` ; nouvel identifiant de session à la connexion
Mots de passe	BCrypt ; 12 caractères minimum, majuscule, minuscule, chiffre ; 3 derniers interdits ; verrouillage 15 min après 5 échecs
Mot de passe temporaire	Filtre bloquant toute action tant que le mot de passe n'est pas changé (RG-05)
Inactivité	Déconnexion après 30 minutes, avertissement à l'écran
Autorisations	Règles par famille d'URL et par rôle ; refus par défaut ; remise gracieuse et administration réservées à ADMIN
Cloisonnement des adhérents	Routes `/me/**` : l'adhérent vient de la session, jamais d'un paramètre (pas d'IDOR)
CSRF	Jeton `XSRF-TOKEN` exigé pour toute requête modifiante
En-têtes	CSP stricte (aucune ressource externe), `nosniff`, `frame-ancestors none`, Referrer-Policy
Données personnelles	Visibles du seul personnel et de l'adhérent concerné ; anonymisation à l'archivage et 4 mois après un retour
Traçabilité	Journal d'audit de toute création ou modification (auteur, date, avant/après, sans mot de passe), conservé 12 mois
Secrets	Uniquement par variables d'environnement ; administrateur initial créé au premier démarrage
Base de données	Deux utilisateurs : migration (DDL) et application (lecture/écriture seulement)

9 Traitements planifiés

Heure (Paris)	Traitement	Règle
00:00	Emprunts échus → en retard	RG-27
00:05	Mises de côté non retirées → expirées, exemplaire réattribué	RG-33, RG-32
00:10	Adhésions échues → expirées	RG-16
02:00	Anonymisation des emprunts rendus depuis 4 mois	RG-19
03:00	Purge du journal d'audit de plus de 12 mois	NFR-08

Chaque traitement est idempotent, travaille par lots de 500 et journalise un résumé. Les lectures n'attendent pas ces traitements : une échéance dépassée est présentée comme un retard dès le lendemain, même avant minuit passé.

10 Conception de l’interface (UX/UI)

10.1 Principes

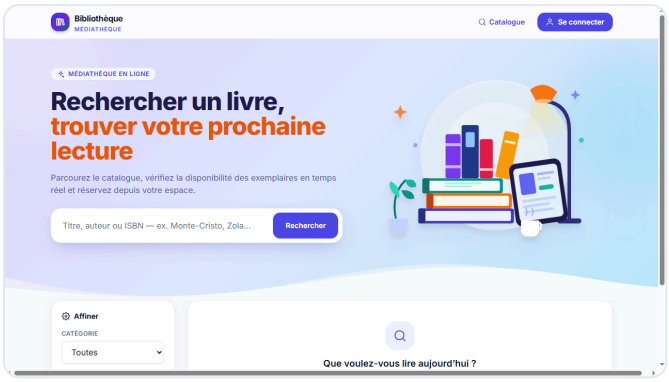
- **Trois gabarits** : public (catalogue), adhérent (onglets « Mon espace »), personnel (menu latéral sombre, raccourcis clavier Alt+E, R, A, C).
- **Accessibilité WCAG 2.1 AA** : navigation au clavier, focus visible, libellés explicites, statut jamais porté par la couleur seule, lien d’évitement.
- **Identité visuelle** : indigo (marque), sarcelle et orange (accents), fonds lavande, police Inter, cartes arrondies, couvertures générées.
- **États d’écran** systématiques : chargement (squelettes), vide, erreur avec « Réessayer », succès.
- **Architecture Angular** : composants standalone, détection OnPush, signals, un dossier par fonctionnalité (`features/*`), composants partagés (`shared/ui`).

10.2 Inventaire des écrans

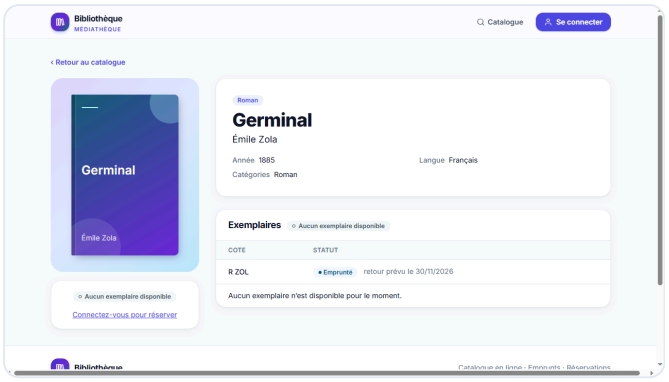
ID	Écran	Fonctionnalité
ECR-01	Page introuvable	socle
ECR-02	Accès refusé	socle
ECR-03	Connexion	authentification
ECR-04	Changer mon mot de passe	authentification
ECR-05	Comptes du personnel	authentification
ECR-06	Dialogue « Nouveau compte du personnel »	authentification
ECR-07	Dialogue « Réinitialiser le mot de passe »	authentification
ECR-08	Recherche dans le catalogue (public)	catalogue
ECR-09	Fiche d'un livre (public)	catalogue
ECR-10	Catalogue (gestion)	catalogue
ECR-11	Formulaire de notice (création / modification)	catalogue
ECR-12	Fiche notice (gestion)	catalogue
ECR-13	Dialogue « Ajouter des exemplaires »	catalogue
ECR-14	Dialogue « Modifier / retirer un exemplaire »	catalogue
ECR-15	Catégories	catalogue
ECR-16	Recherche d'adhérents	adherents
ECR-17	Formulaire adhérent (inscription / modification)	adherents
ECR-18	Fiche adhérent	adherents
ECR-19	Dialogue « Suspendre / réactiver »	adherents
ECR-20	Dialogue « Archiver l'adhérent »	adherents
ECR-21	Adhérents à archiver	adherents
ECR-22	Mon espace (adhérent)	adherents
ECR-23	Mes coordonnées (adhérent)	adherents

ID	Écran	Fonctionnalité
ECR-24	Guichet : emprunt	emprunts
ECR-25	Guichet : retour	emprunts
ECR-26	Emprunts en retard	emprunts
ECR-27	Dialogue « Déclarer un exemplaire perdu »	emprunts
ECR-28	Dialogue « Exemplaire retrouvé »	emprunts
ECR-29	Panneau « Historique de l'exemplaire »	emprunts
ECR-30	Dialogue « Réserver ce livre » (adhérent)	reservations
ECR-31	Dialogue « Réserver pour un adhérent » (personnel)	reservations
ECR-32	Panneau « File d'attente » (fiche notice gestion)	reservations
ECR-33	Réservations à traiter	reservations
ECR-34	Dialogue « Encaisser des pénalités »	penalites
ECR-35	Pénalités impayées	penalites
ECR-36	Dialogue « Accorder une remise gracieuse »	penalites
ECR-37	Tableau de bord du jour	pilotage
ECR-38	Statistiques d'activité	pilotage
ECR-39	Paramètres des règles de prêt	pilotage
ECR-40	Journal d'audit	pilotage

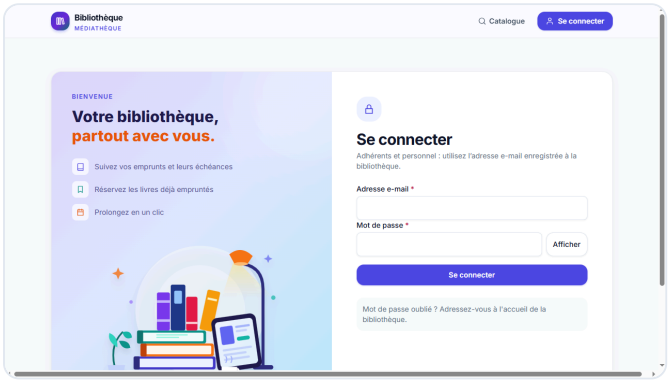
10.3 Captures de l'application



Accueil public du catalogue



Fiche publique d'un livre



Connexion

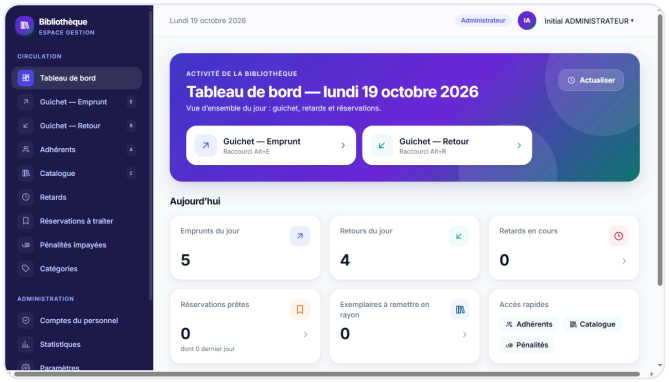
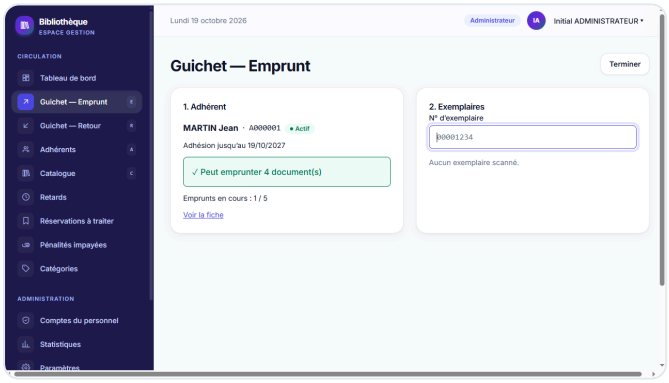
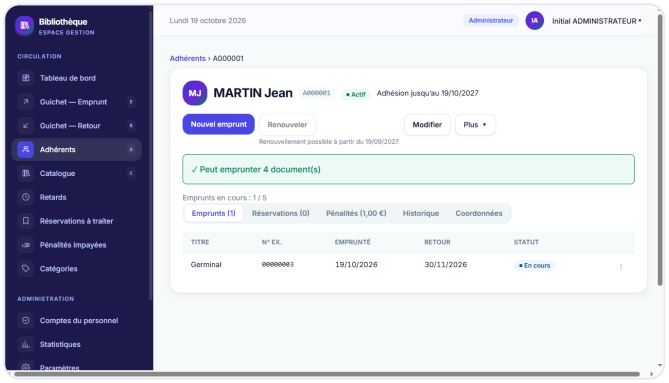


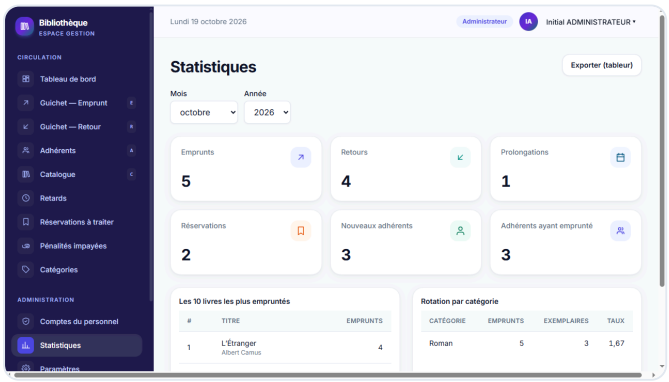
Tableau de bord du personnel



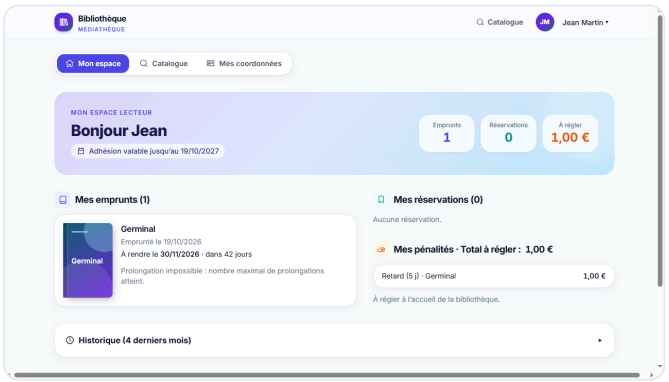
Guichet emprunt



Fiche adhérent



Statistiques (administrateur)



Espace adhérent

11 Déploiement

Service	Image	Rôle	Exposition
db	mariadb:11.4	Base de données, volume persistant	Réseau interne uniquement
backend	bibliotheque-backend	API Spring Boot, migrations Flyway au démarrage	Réseau interne uniquement
frontend	bibliotheque-frontend	nginx : application Angular + relais /api	Seul port publié
db-backup	mariadb:11.4	Sauvegarde quotidienne (mariadb-dump), rotation	Réseau interne

Les images sont construites en plusieurs étapes (build puis exécution sur image minimale, utilisateur non-root). La configuration passe par un fichier `.env` (modèle `.env.example`, jamais versionné). Chaque service a un contrôle de santé ; le backend attend que la base soit prête. Démarrage : `docker compose up -d --build`.

> **Mise en production.** Le service doit être servi en HTTPS (terminaison TLS en amont : reverse-proxy ou hébergeur), car le cookie de session est marqué `Secure`.

12 Stratégie de tests et résultats

Niveau	Outil	Résultat obtenu
Tests unitaires et d'API backend	JUnit 5, Mockito, MockMvc	329 tests, 0 échec ; couverture des services ≥ 98 %
Tests d'architecture	ArchUnit	Conformes (modules et couches)
Tests frontend	Vitest	121 tests, 0 échec
Contraintes SQL	MariaDB réelle	35 cas vérifiés (31 refus attendus, 4 cas valides)
Parcours d'API réel	Script sur pile Docker isolée	86 vérifications sur 86
Parcours navigateur	Cypress	Guichets, fiche adhérent, encaissement, espace adhérent, pilotage : conformes
Sécurité navigateur	Edge sous CSP de production	Aucun blocage de ressource

13 Décisions d’architecture (ADR)

ADR	Décision	Justification
ADR-001	Backend modulaire par domaine (6 modules), couches web/service/repository, vérifié par ArchUnit	Lisibilité, évolutivité, dépendances maîtrisées
ADR-002	MariaDB 11.4 + Flyway ; règles en base (CHECK, UNIQUE, FK) ; unicités partielles par colonnes générées	Intégrité garantie même en cas d’accès concurrent
ADR-003	API REST /api/v1, erreurs ProblemDetail avec codes métier, pagination normalisée, OpenAPI source de vérité	Contrat stable entre frontend et backend
ADR-004	Session serveur (Spring Session JDBC) plutôt que JWT ; CSRF ; cookie strict	Révocation immédiate, pas de jeton exposé au JavaScript
ADR-005	Gradle (Groovy) avec wrapper, images Docker multi-étapes, Docker Compose	Build reproductible, déploiement simple
ADR-006	Traitements planifiés Spring idempotents, par lots, sur le fuseau Europe/Paris	Exactitude des échéances et des pénalités

14 Traçabilité et limites

14.1 Couverture des user stories

Domaine	User stories	Livrées
Authentification et comptes utilisateurs	US-001, US-002, US-003, US-004, US-005, US-006	6 / 6
Catalogue et exemplaires	US-007, US-008, US-009, US-010, US-011, US-012, US-013, US-014, US-015	8 / 9
Adhérents et espace adhérent	US-016, US-017, US-018, US-019, US-020, US-021, US-022, US-023, US-024	9 / 9
Emprunts, retours et prolongations	US-025, US-026, US-027, US-028, US-029, US-030, US-031	7 / 7
Réservations	US-032, US-033, US-034, US-035, US-036	4 / 5
Pénalités	US-037, US-038, US-039, US-040	4 / 4
Pilotage : tableau de bord, statistiques, paramétrage et audit	US-041, US-042, US-043, US-044	4 / 4

14.2 Écarts et limites connus

- **US-036** (e-mail « réservation prête », COULD) : reportée, aucun service d'envoi d'e-mails n'est configuré.
- **US-015** (pré-remplissage par ISBN, COULD) : non livrée dans cette version.
- Les étapes de revue de code et de recette QA (Testcontainers, Cypress complet) restent à conduire.
- Les couvertures de livres sont générées par l'application (aucune image externe, conformément à la CSP).